ADR-001 — Song server architecture

- **Status:** Accepted
- **Date:** 2026-09-05
- **Deciders:** Jay
- **Supersedes:** none

Context

YARG is an LGPL-3.0-or-later Unity/C# rhythm game. It has no server component: every client scans its own local songs folder and builds a private `songcache.bin`. Sharing a library across machines today means copying folders by hand.

We want a self-hosted song server — Docker-first, portable to Raspberry Pi, macOS and Windows — that serves a shared library to YARG clients, with a longer-term goal of LLM-generated charts.

Four decisions had to be made before any code:

1. Repository layout
2. Server implementation language
3. How the client eventually consumes the server
4. Where the public mirror lives

Decision

1. Two repositories, not a monorepo

- `yarg-song-server` — a clean new repo with no upstream history.
- `yarg` — a true fork of `YARC-Official/YARG`, carrying `upstream` as a remote.

Why: the client fork is a 3,800-commit Unity tree with Git LFS and a submodule, and it must stay cleanly rebasable on upstream `dev` for PRs to be acceptable. Putting fast-moving server commits on top of that makes every upstream PR messier and slows server iteration for no gain. The two artifacts have different release cadences, different build toolchains, and different audiences.

2. Go for the server

Why: a single static binary cross-compiles to `linux/arm64` (Pi), macOS and Windows from one source tree, with a container measured in tens of megabytes and no runtime to install. For something meant to run unattended on a Pi in someone's living room, that is the dominant concern.

What it costs, stated plainly: YARG.Core is a standalone .NET library that already does chart parsing, song scanning and metadata extraction. Choosing Go means giving that up and reimplementing the parts we need, with a permanent risk of drift from upstream's behaviour.

Why that cost is acceptable: the research (`docs/research/yarg-song-formats.md`) established that the surface a server actually needs is small and stable:

- Song identity is `SHA1(chart file bytes)` — nothing else. Not the folder, not the audio. That is ~10 lines of Go and it makes client/server agreement on "do I have this song" exact rather than approximate.
- The two formats that carry essentially all community content — loose folders and `.sng` — are both straightforward: `song.ini` is a lenient key/value file, and `.sng` is an uncompressed container with a 16-byte XOR mask and fixed-offset headers.
- The genuinely hard parts of YARG.Core (full MIDI chart semantics, HOPO inference, Phase Shift SysEx) are things **a catalog server has no reason to do at all**. The client already has YARG.Core; duplicating it would be waste, not parity.

The drift risk is real and is mitigated by round-trip testing against a real YARG install rather than by shared code.

Rejected alternatives: .NET/ASP.NET Core (would have shared YARG.Core, but a heavier runtime on the Pi and a worse container story); Python/FastAPI (familiar, best LLM ecosystem for phase 5, but needs a second runtime for parsing and the worst deployment story of the three); a hybrid .NET-parser + Python-API (two runtimes and an IPC boundary to maintain for one project).

3. Sync client first, native client integration second

Phase 2 ships a companion binary that pulls the library into an ordinary local songs folder, so an **unmodified** YARG works. Phase 3 then adds a native remote song source to the fork.

Why: it decouples "is the server correct?" from "is the client change acceptable to upstream?" The sync path proves the server end-to-end in days with zero client risk, and it means the project is useful to other people long before any fork work lands. It also matches the precedent upstream has already set: the Official Setlist is delivered out-of-band by the YARC Launcher, not by the game.

4. Vault2 GitLab is origin; GitHub is a downstream mirror

`gitlab.badassium.com/fatalexception/*` is the source of truth. `github.com/coffeehedake` is a one-way push mirror. Nothing is ever pushed to GitHub directly.

5. LGPL-3.0-or-later for the server

Even though a separately-distributed Go server that links no YARG code and merely speaks a network protocol would not be forced to it — file formats are not copyrightable, and LGPL, unlike AGPL, does not reach across a network boundary.

Why choose it anyway: the project's stated goal is eventual upstream contribution. Matching upstream's licence removes all ambiguity, keeps code movable in both directions, and costs nothing for a project that is open-source regardless. It also covers the case where a parser ends up being a close port of LGPL source, which would trigger the obligation anyway.

Consequences

Good

- Server iteration is not gated on a Unity fork; client PRs stay clean against upstream `dev`.
- One `go build` produces every target platform; the Pi story is free rather than an afterthought.
- Sharing a hash definition with YARG exactly means dedup and "what am I missing" are precise.
- Useful output exists before any client modification, so the upstream conversation can be opened with a working thing rather than a proposal.

Bad / accepted

- Format parsing is duplicated and can drift from YARG.Core. Mitigated by round-tripping against a real YARG install in CI, not by hoping.
- Rock Band CON content cannot be ingested, ever — mogg decryption is out of scope upstream and legally fraught. Users with RB libraries are not served by this project. **Enforced since 2026-09-06:** `.con`, `_rb3con`, `.pkg` and `.xex` are recognised and refused with a stated reason rather than ignored, because a file that vanishes silently reads as a broken server. See ADR-003.
- Phase 5's LLM work will want Python tooling; it will have to talk to the Go server over a boundary rather than living inside it.
- Two repos means two CI configs, two release processes, and cross-repo changes need coordination.

Notes

- Upstream's `CONTRIBUTING.md` does not mention networking, remote song sources or a server in any of its six scope tiers. That is an absence, not approval. Open a discussion before building the Phase 3 PR.
- `.yargsong` does not exist. `.yarground` is a Unity AssetBundle venue/background and can only be passed through as an opaque blob.
